

Now, **Question: "Does Python have types in each pillar like Java?"**

The short answer is **Yes, but Python does it very differently than Java.**

Java is a "strict" language. It uses strict keywords and forces you to obey the rules. Python is a "dynamic" language. It treats you like a responsible adult and relies on *conventions* and flexibility.

If you are transitioning from Java, or if a recruiter asks you to compare them, here are the exact "Types" within the 4 pillars in Python.

1. Types of Encapsulation (Access Modifiers)

In Java, you have strict keywords: `public`, `protected`, and `private`. If you try to access a private variable in Java, the code literally will not compile.

Python does not have these keywords. It uses **Underscores**:

- **Public (No underscores):** `self.name`
 - Anyone can see and change this.
 - **Protected (Single underscore):** `self._name`
 - *Java equivalent:* Accessible only in the class and its children.
 - *Python reality:* Python won't actually stop you from touching it. It is just a "gentleman's agreement" among programmers that means *"Please do not touch this outside the class."*
 - **Private (Double underscore):** `self.__name`
 - *Java equivalent:* Strictly hidden.
 - *Python reality:* Python uses a trick called **Name Mangling** to change the variable's name behind the scenes so outside code can't easily find it.
-

2. Types of Inheritance (Single vs. Multiple)

This is a massive difference between the two languages.

- **Java:** Only supports **Single Inheritance**. A child can only have ONE parent class. (Java uses "Interfaces" to fake having multiple parents).

- **Python:** Fully supports **Multiple Inheritance** natively! A child class can have two, three, or ten actual parent classes.

Python Multiple Inheritance Example:

```
class Bird:
    def fly(self):
        print("Flying...")

class Machine:
    def use_electricity(self):
        print("Bzzzt...")

# The Child has TWO parents!
class Drone(Bird, Machine):
    pass

my_drone = Drone()
my_drone.fly()           # Inherited from Bird
my_drone.use_electricity() # Inherited from Machine
```

*(Pro Interview Tip: If an interviewer asks what happens if both parents have a method with the same name, tell them Python uses **MRO (Method Resolution Order)**, which reads parents from left to right!)*

3. Types of Polymorphism (Overriding vs. Overloading)

In Java, there are two famous types of Polymorphism:

1. **Overriding:** A child class replaces its parent's method.
2. **Overloading:** Writing the *same method name* multiple times in the same class, but with different parameters.

Does Python have this?

- **Overriding:** YES. Python uses this heavily (especially in AI/PyTorch).
- **Overloading:** NO. Python does NOT support traditional Method Overloading. If you write two methods with the same name in Python, the bottom one just deletes the top one.

Instead of overloading, Python uses **Default Arguments**:

```
# In Java, you would write 3 different 'greet' methods.  
# In Python, we write ONE method and use default values (None):  
class Greeter:  
    def greet(self, name=None):  
        if name is None:  
            print("Hello, Stranger!")  
        else:  
            print(f"Hello, {name}!")
```

Python also uses something famous called "**Duck Typing**" (If it walks like a duck and quacks like a duck, it's a duck). Python doesn't care what type an object is, as long as it has the right method.

4. Types of Abstraction (Abstract Classes vs. Interfaces)

In Java, you have the `interface` keyword and the `abstract` keyword.

Python does not have an `interface` keyword. To do strict Abstraction in Python, we have to import a built-in library called **abc (Abstract Base Classes)**.

When you use `abc`, you can force developers to write certain methods. If they don't, Python will crash the program.

```
from abc import ABC, abstractmethod

# This is an Abstract Class. You cannot build an object out of this!
# It is purely a template that forces rules on its children.
class AIModel(ABC):

    @abstractmethod
    def train(self):
        pass # We leave it blank. The child MUST fill this in.

# If we build a child class but forget to write the 'train' method...
class ChatBot(AIModel):
    def talk(self):
        print("Hello!")

# ...Python will throw an ERROR here and refuse to build the bot!
my_bot = ChatBot()
# TypeError: Can't instantiate abstract class ChatBot with abstract
method train
```
